

Using generative AI to visualize change in DaRTS

Lab Goal

In this lesson we are going to visualize change in the DaRTS dataset, using generative AI to assist with developing the code.

Lab Outcomes

1. Learn how to effectively use generative AI for developing code (e.g., appropriate prompts, checks on output)
2. Develop code to visualize changes over time or location for oceanographic data.

Introduction

Generative AI

Generative AI (genAI) is very good at writing `ggplot2` boilerplate quickly, and very bad at knowing what your data actually looks like unless you tell it. Throughout this tutorial you'll see boxes like this:

Try this prompt: *“I have an R data frame called `ocean_data` with columns `date`, `station`, `depth` (meters), and `temperature` (°C). Write `ggplot2` code to plot a depth profile of temperature for a single date, faceted by station, with depth increasing downward on the y-axis.”*

Copy a prompt like that into your AI assistant of choice, run the code it gives you, and then compare it to the worked example immediately below. They won't be identical — that's fine. The goal is to notice *what information in the prompt shaped the code you got back*, and to build the habit of always specifying: **what your columns mean, what plot type you want, and what the axes/encoding should be.**

Visualizing Change

In our initial exploration of the data, we've looked at how to plot some variables under some limited conditions e.g. profiles for one cruise, or relationships between a couple of variables for all cruises. We could have used Excel to do this plotting. But what about if we want to visualize our data in a different way that represented the time and space components of field data or if we wanted a more complicated subset of our data? That's where R comes in really handy.

We will work through a few different examples using the DaRTS combined data csv file “DaRTS_combined_data.csv”, where we are using genAI in different ways.

But, it's worth pointing out here that all the methods we'll be using can be used on any type of data - and we'll see some examples as we go through the tutorial.

Exercise 1 - Time series of surface data

Let's start with the following question:

How does surface chlorophyll fluorescence change at each station over the 2016 sampling season?

Surface can be defined in multiple ways and the exact choices you make will depend on your data and your situation. Here, we are going to say surface chlorophyll fluorescence is the average value over the top 5 m.

Another example: Say you've got three cultures that you're testing four different treatments on. For each culture/treatment combination, you're measuring cell counts at a series of time points. What you'd like to visualize/plot is the average cell counts over the first two time points for each culture and treatment.

How do the variables in these examples compare?

- station is equivalent to treatment
- date is equivalent to culture
- depth is equivalent time point

Going back to the chlorophyll fluorescence example - how would you visualize this? Let's try a line plot, with a different line for each station, chlorophyll fluorescence on the y-axis, date on the x-axis.

If you have an idea of your final figure, you can give the genAI those details. But first, we need to tell the genAI about the dataset we're using.

Try this prompt: *"I have an R data frame containing oceanographic data called **darts** with many columns. We're going to do some data manipulation as necessary and make figures using ggplot2."*

From this command, I'm being prompted to provide more details about my dataset to the AI. So let's import our dataset.

```
darts <- read.csv("../data/DaRTS_combined_data.csv")
```

I was asked for the output of `str(darts)` or `head(darts)` - so let's generate those:

```
str(darts)
```

```
## 'data.frame':    17196 obs. of  37 variables:
## $ X.1              : int  1 2 3 4 5 6 7 8 9 10 ...
## $ date             : chr  "2012-09-11" "2012-09-11" "2012-09-11" "2012-09-11" ...
## $ station          : int  1 1 1 1 1 1 1 1 1 1 ...
## $ depth_m          : num  1 2 3 4 5 6 7 8 9 10 ...
## $ temp_c           : num  16.9 16.6 16.3 16.2 16.1 ...
## $ sal_psu          : num  31.6 31.6 31.7 31.7 31.7 ...
## $ density_kg_per_m3 : num  22.9 23 23.1 23.2 23.2 ...
## $ par              : num  1876 1270 754 496 302 ...
## $ fluor_mg_per_m3   : num  2.81 2.96 4.28 4.84 4.7 ...
## $ turbidity_ntu     : num  1.3 1.37 1.33 1.13 1.19 ...
## $ beamc_per_m       : num  NA NA NA NA NA NA NA NA NA ...
## $ O2_umol_per_kg    : num  225 246 244 241 239 ...
## $ O2_pct_sat        : num  89.7 97.8 96.5 95.2 94.1 ...
## $ ph                : logi  NA NA NA NA NA NA ...
```

```
## $ X : logi NA NA NA NA NA NA ...
## $ year : int 2012 2012 2012 2012 2012 2012 2012 2012 2012 2012 ...
## $ month : int 9 9 9 9 9 9 9 9 9 9 ...
## $ day : int 11 11 11 11 11 11 11 11 11 11 ...
## $ doy : int 255 255 255 255 255 255 255 255 255 255 ...
## $ cruise : chr NA "1" NA NA ...
## $ alk : logi NA NA NA NA NA NA ...
## $ SiO4_uM : num NA NA NA NA NA NA NA NA NA NA ...
## $ NO3_2_uM : num NA 0 NA NA NA NA NA NA NA 0 ...
## $ PO4_uM : num NA 0.49 NA NA NA NA NA NA NA 0.56 ...
## $ NH4_uM : num NA 0.05 NA NA NA NA NA NA NA 0.02 ...
## $ total_pico_cells_per_ml : int NA 5618 NA NA NA NA NA NA NA 6299 ...
## $ pico_plankton_cells_per_ml : num NA NA NA NA NA NA NA NA NA NA ...
## $ bacteria_cells_per_ml : num NA 1870000 NA NA NA NA NA NA NA 2330000 ...
## $ viral_cells_per_ml : num NA 45100000 NA NA NA NA NA NA NA 44700000 ...
## $ chl_tot_ug_per_l : num NA 9.12 NA NA NA ...
## $ chl_lt20um_ug_per_l : num NA 7.93 NA NA NA ...
## $ chl_lt3um_ug_per_l : num NA 0.222 NA NA NA NA NA NA NA ...
## $ flowcam_biomass : int NA 224000 NA NA NA NA NA NA NA ...
## $ cyanobacteria_cells_per_ml : int NA 722 NA NA NA NA NA NA NA 574 ...
## $ picoeukaryotes_cells_per_ml : int NA 3817 NA NA NA NA NA NA NA 4365 ...
## $ noeukaryotes_cells_per_ml : int NA 1079 NA NA NA NA NA NA NA 1361 ...
## $ cryptophytes_cells_per_ml : int NA NA NA NA NA NA NA NA NA ...
```

OK now we can go back to our AI assistant and ask it to make our plot:

Try this prompt: *“I want to plot chlorophyll fluorescence from the surface (y-axis) with date (x-axis) and a line for each station, but only for 2016.”*

From this prompt, my AI assistant gave me two chunks of code, one where it took “surface” to be the average over the top 5m, and one where it gave me the top depth only, because I didn’t specify what I meant by surface. While in this case, the AI flagged there was some ambiguity in the term surface, it may have just made an assumption on what I meant and not told me - I **should** have provided how I wanted to define “surface” in my prompt. A better prompt would have been:

Try this prompt: *“I want to plot chlorophyll fluorescence from the surface (y-axis) with date (x-axis) and a line for each station, but only for 2016. Define surface as the mean of the data over depths less than 5m”*

Below is the code we got from the above prompts. Let’s look closely at what’s it’s doing. We want to check:

1. Did it use the correct columns? In our prompt we didn’t give it the column headers exactly as they appear in the dataset. In my case, the AI used the correct columns. But, it’s good practise to use the exact column headers in the prompt.
2. Did it use the correct units, labels, etc?
3. Did it do any data manipulation I wasn’t expecting? In my case, the AI told me it would average over the top 5m, so I’m looking out for that manipulation. But if it hadn’t told me it was doing that, I would have been wondering what that piece of code was doing. If you are unsure about some of the code it returns, you can always ask it what specific lines or blocks of code are doing.
4. Is the output of the code correct? e.g., did it make the figure you wanted it to make?

```

library(ggplot2)
library(dplyr)

##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

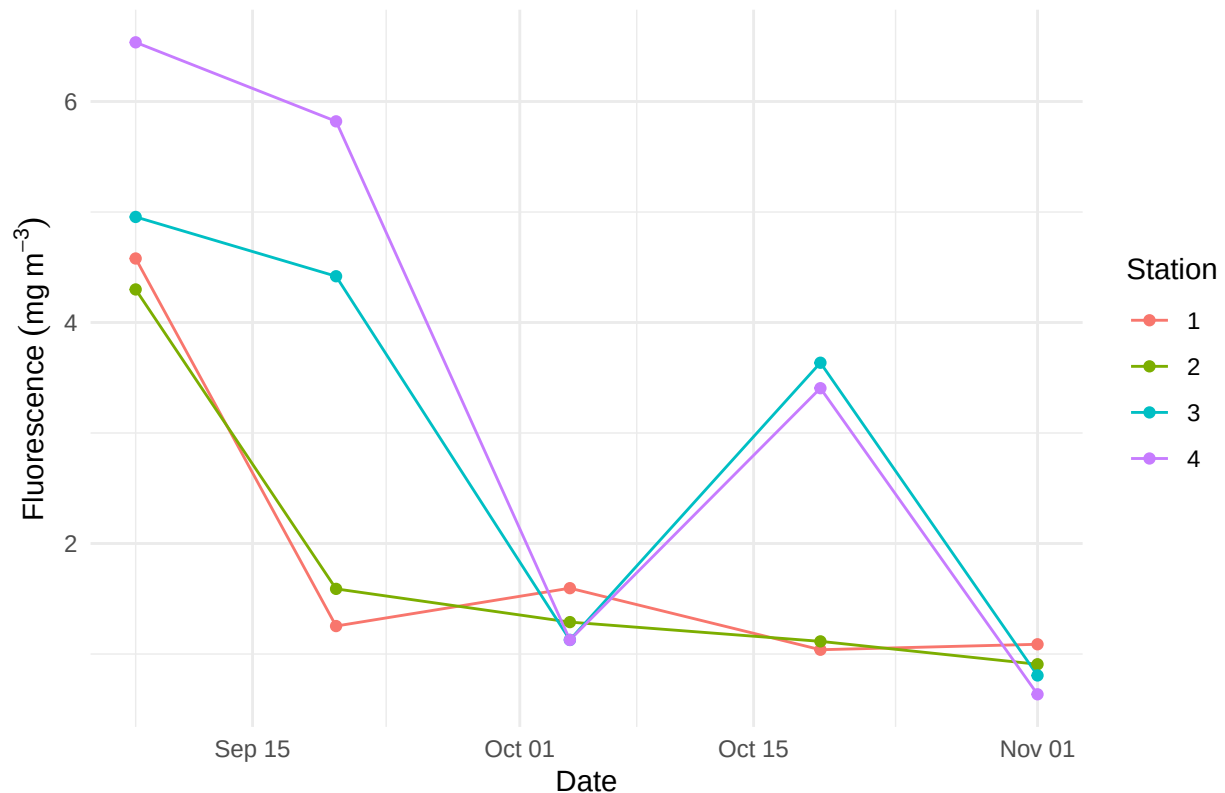
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union

darts_surface_2016 <- darts %>%
  filter(year == 2016, depth_m <= 5) %>%
  group_by(date, station) %>%
  summarize(fluor_mg_per_m3 = mean(fluor_mg_per_m3, na.rm = TRUE), .groups = "drop") %>%
  mutate(
    date = as.Date(date),
    station = factor(station)
  )

ggplot(darts_surface_2016, aes(x = date, y = fluor_mg_per_m3, color = station)) +
  geom_line() +
  geom_point() +
  labs(
    x = "Date",
    y = expression(Fluorescence~(mg~m^{-3})),
    color = "Station",
    title = "Surface Chlorophyll Fluorescence by Station, 2016"
  ) +
  theme_minimal()

```

Surface Chlorophyll Fluorescence by Station, 2016



In this case, the output looks good. But may we want to tweak it a little bit. If you know the commands yourself, you could directly add them to the block of code above. Or you could ask you assistant to update the figure e.g.,

Try this prompt: “I want to make the same plot as above, with surface defined as the average over 5m, but with the lines thicker and colors appropriate for colorblind people (can also use different markers and linestyles to help).”

```

darts_surface_2016 <- darts %>%
  filter(year == 2016, depth_m <= 5) %>%
  group_by(date, station) %>%
  summarize(fluor_mg_per_m3 = mean(fluor_mg_per_m3, na.rm = TRUE), .groups = "drop") %>%
  mutate(
    date = as.Date(date),
    station = factor(station)
  )

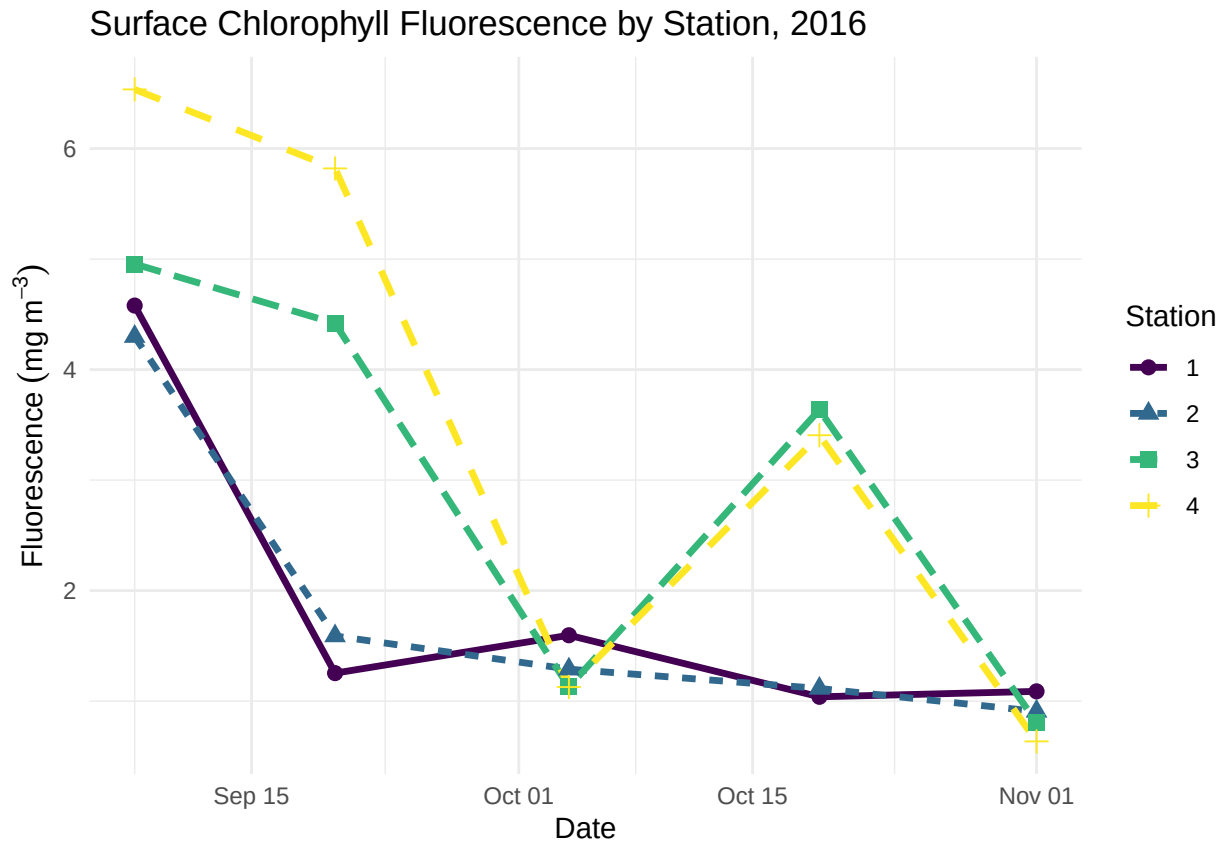
ggplot(darts_surface_2016, aes(x = date, y = fluor_mg_per_m3,
                              color = station, shape = station, linetype = station)) +
  geom_line(linewidth = 1.2) +
  geom_point(size = 2.5) +
  scale_color_viridis_d(option = "viridis") +
  labs(
    x = "Date",
    y = expression(Fluorescence~(mg~m^{-3})),
    color = "Station",
  )

```

```

shape = "Station",
linetype = "Station",
title = "Surface Chlorophyll Fluorescence by Station, 2016"
) +
theme_minimal()

```



Is there another way we could visualize this data? Let's ask the AI for options:

Try this prompt: *"Is there another way I can plot these three variables (surface chlorophyll fluorescence, date and station) to show differences over time and location?"*

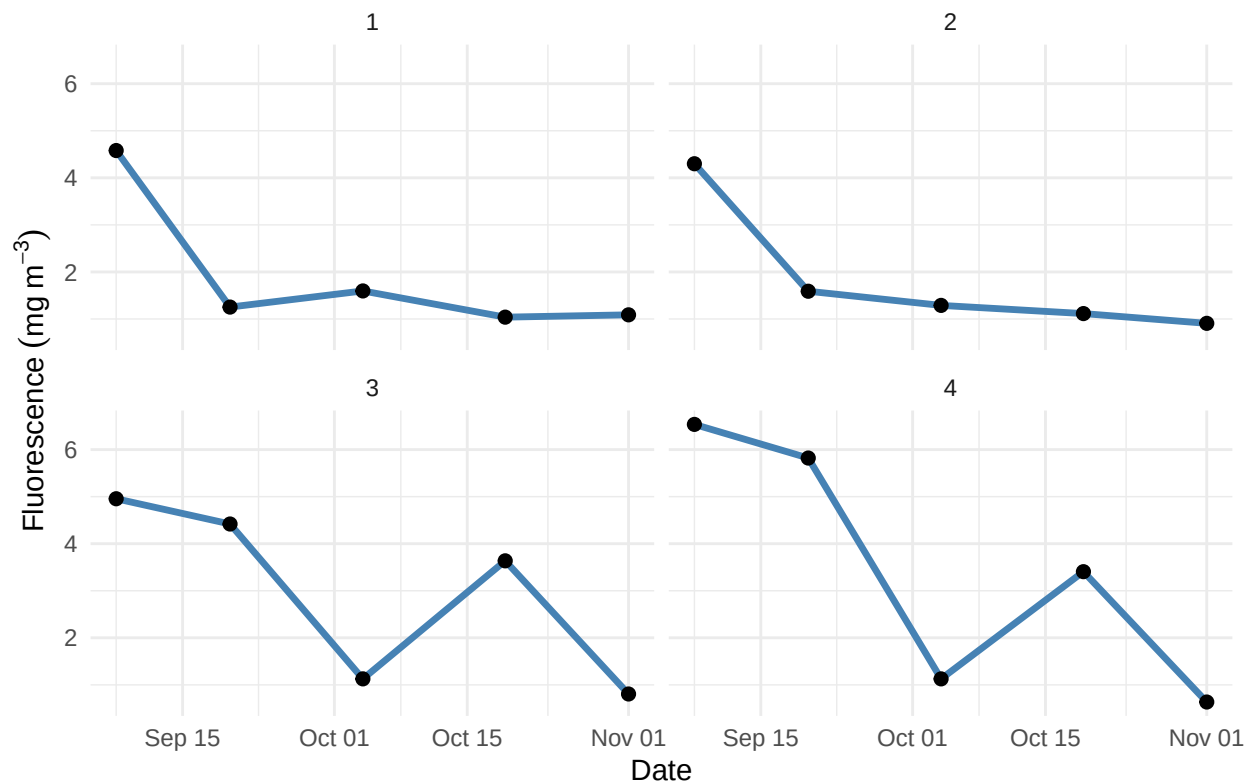
In my case, I got a few options. The first was to facet the above into different subplots:

```

ggplot(darts_surface_2016, aes(x = date, y = fluor_mg_per_m3)) +
  geom_line(linewidth = 1.2, color = "steelblue") +
  geom_point(size = 2) +
  facet_wrap(~ station, ncol = 2) +
  labs(x = "Date", y = expression(Fluorescence~(mg~m^{-3}))),
  title = "Surface Chlorophyll Fluorescence by Station, 2016" +
  theme_minimal()

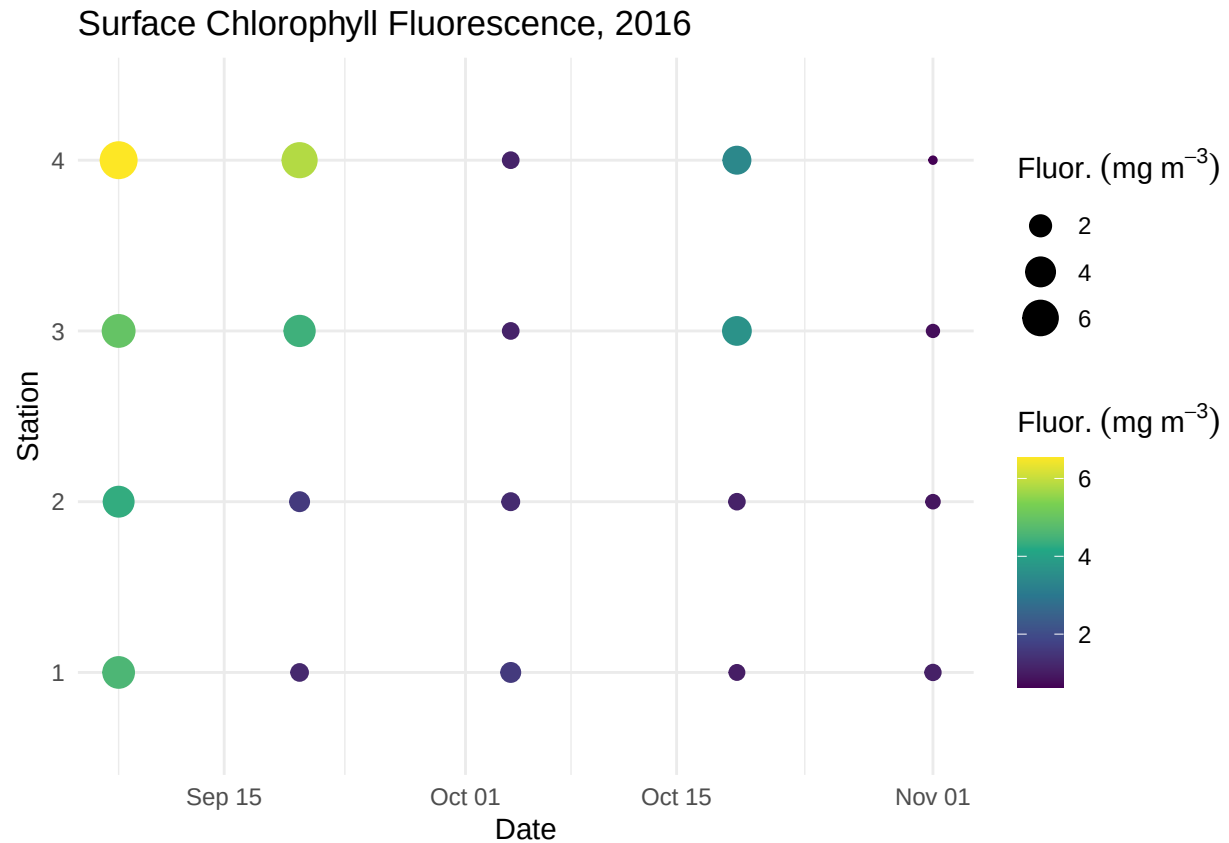
```

Surface Chlorophyll Fluorescence by Station, 2016



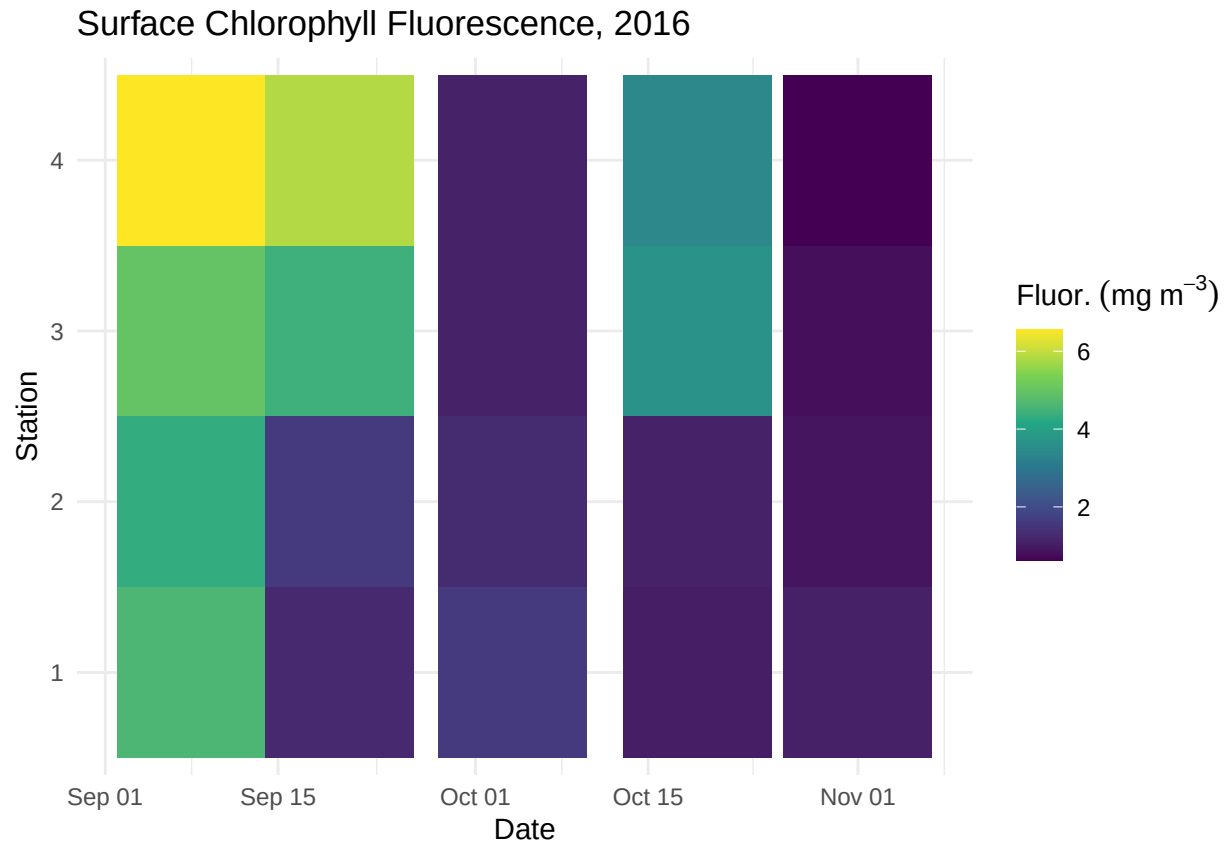
Next, it suggested a bubble plot, which it told me works well if the data is irregularly sampled and there are gaps.

```
ggplot(darts_surface_2016, aes(x = date, y = station, size = fluor_mg_per_m3, color = fluor_mg_per_m3))
  geom_point() +
  scale_color_viridis_c() +
  labs(x = "Date", y = "Station", size = expression(Fluor.~(mg~m^{-3})),
       color = expression(Fluor.~(mg~m^{-3})),
       title = "Surface Chlorophyll Fluorescence, 2016") +
  theme_minimal()
```



Finally, it suggested a heatmap. But note it told me this “works especially well if sampling dates are roughly consistent across stations (irregular sampling leaves gaps)”

```
ggplot(darts_surface_2016, aes(x = date, y = station, fill = fluor_mg_per_m3)) +
  geom_tile() +
  scale_fill_viridis_c(name = expression(Fluor.~(mg~m-3))) +
  labs(x = "Date", y = "Station",
       title = "Surface Chlorophyll Fluorescence, 2016") +
  theme_minimal()
```



Hmmm, this looks OK, but we can see the irregular sampling in terms of date does cause gaps. Can we fill those gaps in somehow?

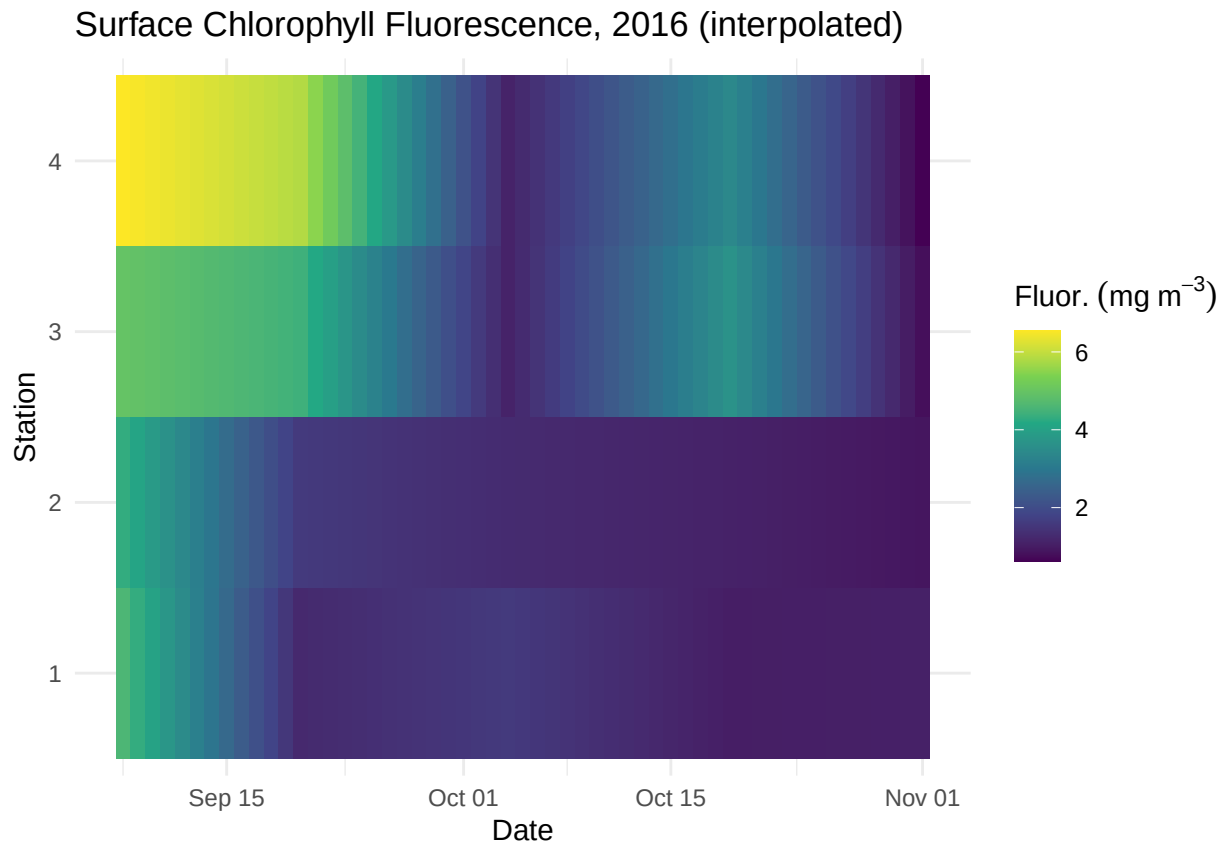
Try this prompt: *“Let’s improve on the heatmap plot - my data is irregularly sampled. Can we fill in the gaps somehow?”*

Again, I got a few options. The first option was for “linear interpolation along a regular date grid, per station: for each station, interpolate fluorescence onto a common daily (or weekly) date sequence using `approx()`, then plot the interpolated grid as a continuous heatmap.” The other options built on this idea. While this might make a nice figure, is this really what I want? What it’s saying is it won’t just fill the gaps between the measured dates, it’s going to make a set of regularly spaced dates between the first sampling date and the last, and then approximate the data onto those regularly spaced dates from the dates where we have measured data. Depending on our question, or the purpose of our figure, we may be OK with this. Let’s see what it looks like:

```
# regular date grid spanning the 2016 sampling period
date_grid <- seq(min(darts_surface_2016$date), max(darts_surface_2016$date), by = "day")

darts_interp <- darts_surface_2016 %>%
  group_by(station) %>%
  arrange(date) %>%
  reframe(
    fluor_mg_per_m3 = approx(x = date, y = fluor_mg_per_m3, xout = date_grid, rule = 2)$y,
    date = date_grid
  )
```

```
ggplot(darts_interp, aes(x = date, y = station, fill = fluor_mg_per_m3)) +
  geom_tile() +
  scale_fill_viridis_c(name = expression(Fluor.~(mg~m-3))) +
  labs(x = "Date", y = "Station",
       title = "Surface Chlorophyll Fluorescence, 2016 (interpolated)") +
  theme_minimal()
```



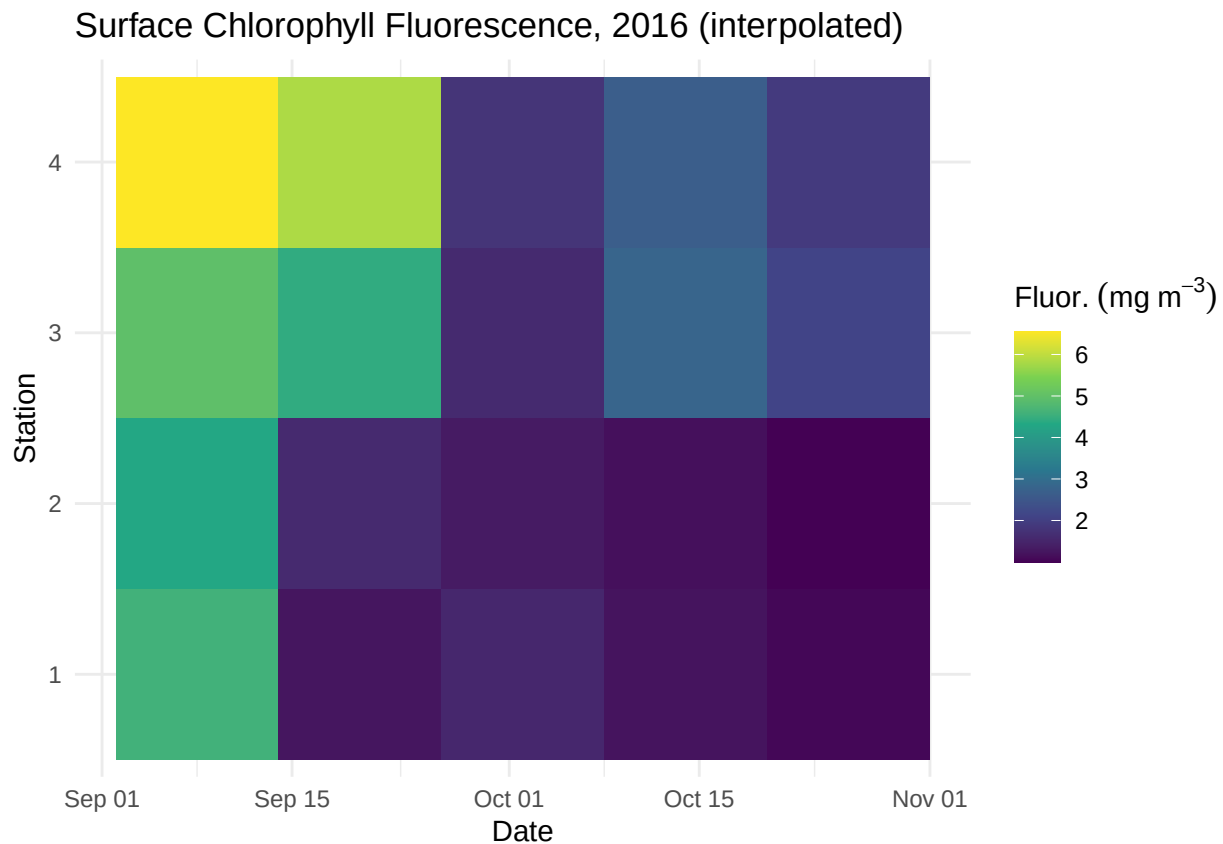
A daily interpolation seems like it's a bit misleading, but maybe if we did it over a longer period. Let's tweak the code above. I settled on interpolating over a 12 day period, because that results in 5 dates over our time period, and we have data from 5 cruises, so we're getting data interpolated very close to our actual measurement dates.

```
# regular date grid spanning the 2016 sampling period
date_grid <- seq(min(darts_surface_2016$date), max(darts_surface_2016$date), by = 12)

darts_interp <- darts_surface_2016 %>%
  group_by(station) %>%
  arrange(date) %>%
  reframe(
    fluor_mg_per_m3 = approx(x = date, y = fluor_mg_per_m3, xout = date_grid, rule = 2)$y,
    date = date_grid
  )

ggplot(darts_interp, aes(x = date, y = station, fill = fluor_mg_per_m3)) +
  geom_tile() +
  scale_fill_viridis_c(name = expression(Fluor.~(mg~m-3))) +
```

```
labs(x = "Date", y = "Station",
     title = "Surface Chlorophyll Fluorescence, 2016 (interpolated)") +
theme_minimal()
```



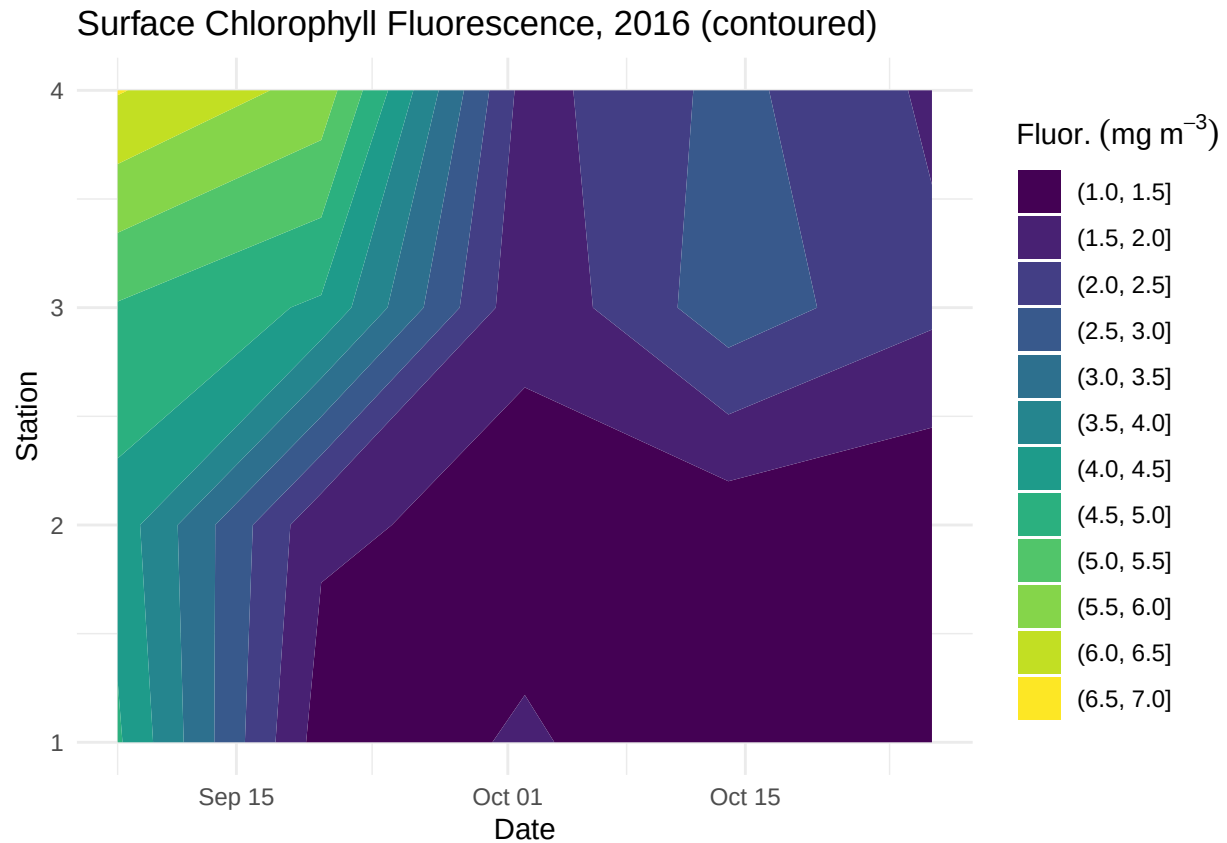
This looks better, but it's still not a great representation of our data. Our AI assistant suggested `geom_tile` because it is the more common answer. But, I know that a contour plot is appropriate for this type of data. So let's ask our AI assistant to generate code for that:

Try this prompt: *"What about using `geom_contour` or similar rather than `geom_tile`?"*

The response I got was that this is a good type of plot for this data, as long as there is a spatial connection between the stations as data will be interpolated along both the x and y axes. For the DaRTS data, having the stations ordered 1 - 4 is how they are spatially connected, so this is OK for us. And here's the result:

```
darts_interp_numeric <- darts_interp %>%
  mutate(station_num = as.numeric(as.character(station)))

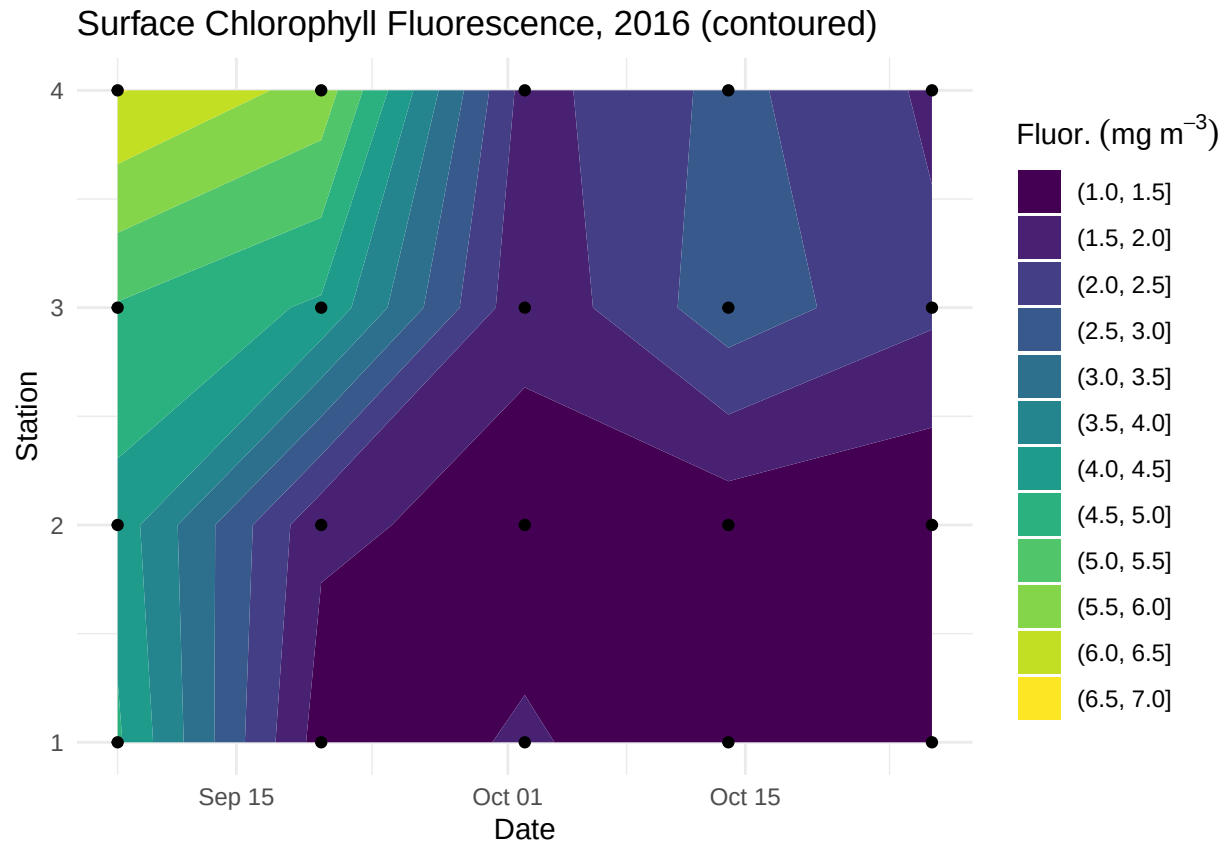
ggplot(darts_interp_numeric, aes(x = date, y = station_num, z = fluor_mg_per_m3)) +
  geom_contour_filled() +
  scale_y_continuous(breaks = sort(unique(darts_interp_numeric$station_num))) +
  scale_fill_viridis_d(name = expression(Fluor.~(mg~m^{-3}))) +
  labs(x = "Date", y = "Station",
       title = "Surface Chlorophyll Fluorescence, 2016 (contoured)") +
  theme_minimal()
```



This looks much better than the `geom_tile` approaches above. But we can make it even better by adding on points to show where the actual measurements were made, so we can see which parts are the interpolated data.

```
darts_interp_numeric <- darts_interp %>%
  mutate(station_num = as.numeric(as.character(station)))

ggplot(darts_interp_numeric, aes(x = date, y = station_num, z = fluor_mg_per_m3)) +
  geom_contour_filled() +
  scale_y_continuous(breaks = sort(unique(darts_interp_numeric$station_num))) +
  scale_fill_viridis_d(name = expression(Fluor.~(mg~m-3))) +
  labs(x = "Date", y = "Station",
       title = "Surface Chlorophyll Fluorescence, 2016 (contoured)") +
  theme_minimal() +
  geom_point()
```



As mentioned above, in this case we interpolated between our stations as well as our dates. But our stations are not equally spaced (in the real world), so a truer representation of our data would be to have the stations spaced as they are in the real world. To do this, we'll use the latitude of each station, rather than the station number.

Station numbers and corresponding latitudes: for stations [1, 2, 3, 4], their latitudes are [43.903, 43.863, 43.811, 43.753].

Try this prompt: *“Can we generate the above `geom_contour_filled` plot, but rather than use station numbers on the y-axis, use their latitudes. for stations [1, 2, 3, 4], their latitudes are [43.903, 43.863, 43.811, 43.753]. Also add points where the measurements were made.”*

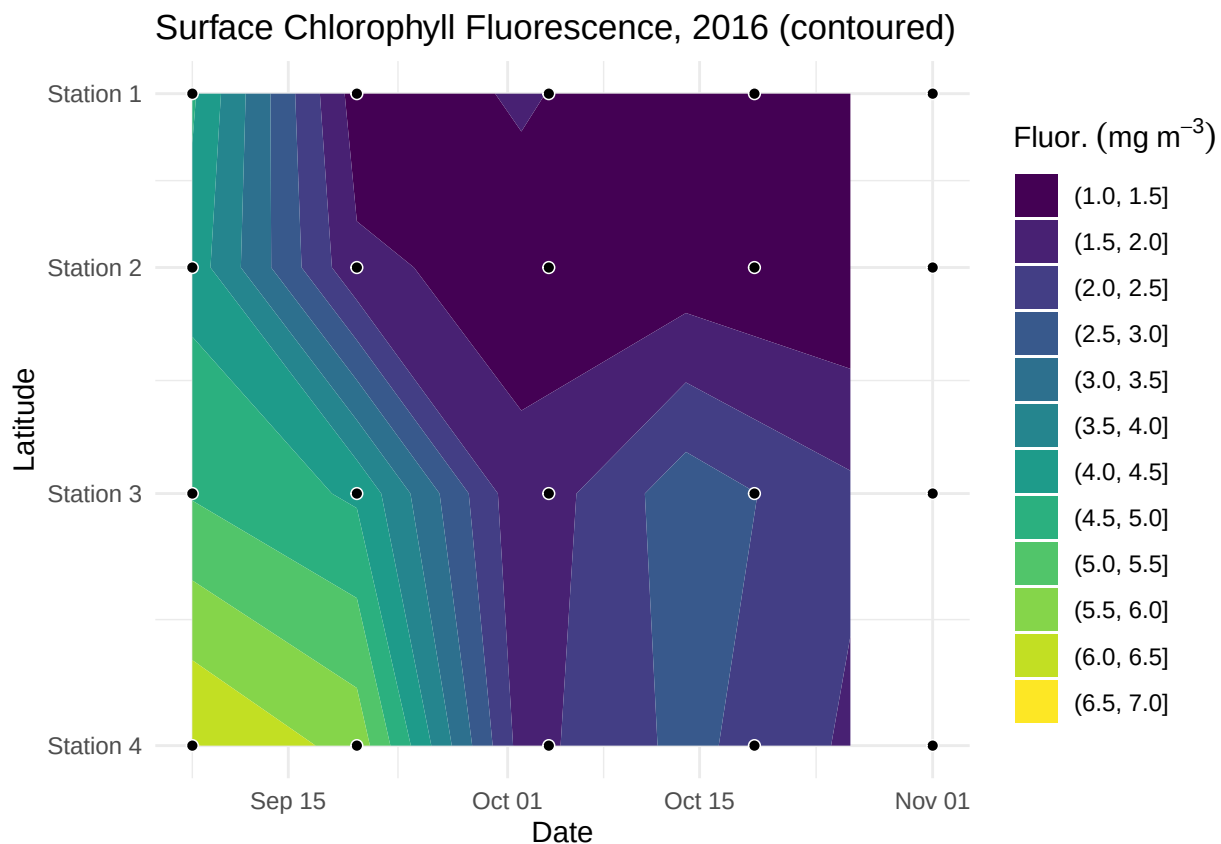
```
# station -> latitude lookup
station_lat <- tibble(
  station = factor(c(1, 2, 3, 4)),
  latitude = c(43.903, 43.863, 43.811, 43.753)
)

# join latitude onto the interpolated grid (for the contour surface)
darts_interp_lat <- darts_interp %>%
  left_join(station_lat, by = "station")

# join latitude onto the ORIGINAL (non-interpolated) surface data (for the points)
darts_points_lat <- darts_surface_2016 %>%
  left_join(station_lat, by = "station")

ggplot() +
```

```
geom_contour_filled(data = darts_interp_lat,
  aes(x = date, y = latitude, z = fluor_mg_per_m3)) +
geom_point(data = darts_points_lat,
  aes(x = date, y = latitude),
  color = "white", shape = 21, fill = "black", size = 1.8, stroke = 0.4) +
scale_fill_viridis_d(name = expression(Fluor. $\cdot$ (mg $\cdot$ m-3))) +
scale_y_continuous(breaks = station_lat$latitude,
  labels = paste0("Station ", station_lat$station)) +
labs(x = "Date", y = "Latitude",
  title = "Surface Chlorophyll Fluorescence, 2016 (contoured)") +
theme_minimal()
```



In this exercise, we've created 3 different types of figures to visual change: line plots (single and facet), bubble plots, and contour plots. Depending on your question, or the purpose of your figure, or the patterns you are trying to highlight, one of these plots will likely be more appropriate than another. We also had a lot of back and forth with the AI assistant to get the output we were looking for (e.g., refining plot aesthetics), but we also used our knowledge of what types of plots are appropriate to generate the code - it was important to look at our output and think about any changes we would need to make, rather than just use the first piece of code we were returned.

Exercise 2 - Changes with depth

The figures generated in previous section works well when your data are consistent in terms of the variables you want to compare. For the data we plotted above, there were five different cruises, and on each cruise, data was collected at the same four locations. We had data for every cruise and station. If we'd been missing

data at one of the stations on one of the cruises, we'd just have a blank part on the plots we made. But what if we were missing a lot of data - would the above approach still be a good way to visualize our data?

To dig into this a bit further, let's consider how chlorophyll fluorescence varies by depth at each station *for one cruise*.

In this case, we're going to have station on the x-axis and depth on the y-axis, and we'll consider the cruise that took place on September 8th 2016.

Try this prompt: *"I now want to plot the fluorescence data (color) by depth (y-axis) and latitude (x-axis) for the cruise on Sept 8th 2016"*

I get the following response, which contains a lot of useful details about the code it's generating:

"This is a classic oceanographic section plot (depth vs. distance/latitude along a transect, colored by a property) — a good use case for contouring, since depth is sampled at ~1 m resolution while latitude only has 4 discrete stations, so we're essentially interpolating a smooth curtain between four finely-resolved vertical profiles.

A few differences from the previous plot: we need the full depth-resolved data (not just the surface subset), we filter to a single cruise date, and depth conventionally goes on a reversed y-axis (surface at top, deep at bottom). Since the data is irregular in the latitude direction (4 discrete stations, not a grid), `geom_contour_filled()` needs a proper 2D interpolation first — `akima::interp()` handles this well."

```
library(akima)
```

```
station_lat <- tibble(
  station = factor(c(1, 2, 3, 4)),
  latitude = c(43.903, 43.863, 43.811, 43.753)
)

# full depth-resolved data for the single cruise date
darts_section <- darts %>%
  filter(as.Date(date) == as.Date("2016-09-08")) %>%
  mutate(station = factor(station)) %>%
  left_join(station_lat, by = "station") %>%
  filter(!is.na(fluor_mg_per_m3), !is.na(latitude), !is.na(depth_m))

# 2D interpolation onto a regular latitude x depth grid
interp_grid <- with(darts_section,
  interp(x = latitude, y = depth_m, z = fluor_mg_per_m3,
    duplicate = "mean",
    nx = 100, ny = 100)
)
```

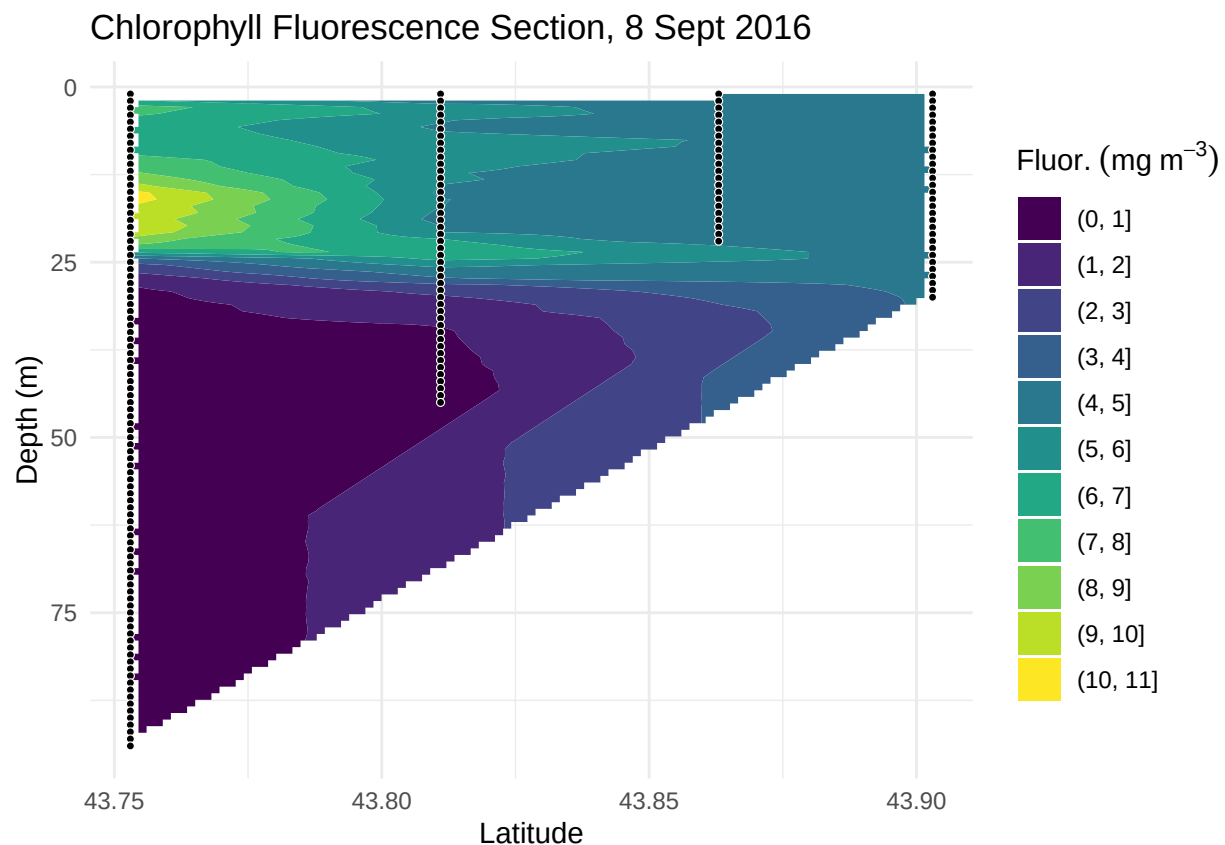
```
## Warning in interp(x = latitude, y = depth_m, z = fluor_mg_per_m3, duplicate =
## "mean", : collinear points, trying to add some jitter to avoid colinearities!
```

```
## Warning in interp(x = latitude, y = depth_m, z = fluor_mg_per_m3, duplicate =
## "mean", : success: collinearities reduced through jitter
```

```
interp_df <- expand.grid(latitude = interp_grid$x, depth_m = interp_grid$y) %>%
  mutate(fluor_mg_per_m3 = as.vector(interp_grid$z))
```

```
ggplot() +
  geom_contour_filled(data = interp_df,
                     aes(x = latitude, y = depth_m, z = fluor_mg_per_m3)) +
  geom_point(data = darts_section,
            aes(x = latitude, y = depth_m),
            color = "white", shape = 21, fill = "black", size = 1.2, stroke = 0.3) +
  scale_y_reverse() +
  scale_fill_viridis_d(name = expression(Fluor.~(mg~m-3))) +
  labs(x = "Latitude", y = "Depth (m)",
       title = "Chlorophyll Fluorescence Section, 8 Sept 2016") +
  theme_minimal()
```

```
## Warning: Removed 3595 rows containing non-finite outside the scale range
## ('stat_contour_filled()').
```



This output looks good, but we have data being interpolated below the seafloor for stations 2 and 3. Let's mask those areas:

Try this prompt: *"Can we mask the areas where interpolation goes below the neighbouring casts? i.e. a pseudo-bathymetry mask"*

```
library(dplyr)
library(ggplot2)
library(akima)
```

```

station_lat <- tibble(
  station = factor(c(1, 2, 3, 4)),
  latitude = c(43.903, 43.863, 43.811, 43.753)
)

darts_section <- darts %>%
  filter(as.Date(date) == as.Date("2016-09-08")) %>%
  mutate(station = factor(station)) %>%
  left_join(station_lat, by = "station") %>%
  filter(!is.na(flour_mg_per_m3), !is.na(latitude), !is.na(depth_m))

# max sampled depth per station (pseudo-bathymetry anchor points)
max_depth_by_station <- darts_section %>%
  group_by(station, latitude) %>%
  summarize(max_depth = max(depth_m, na.rm = TRUE), .groups = "drop") %>%
  arrange(latitude)

# 2D fluorescence interpolation, same as before
interp_grid <- with(darts_section,
  interp(x = latitude, y = depth_m, z = flour_mg_per_m3,
    duplicate = "mean",
    nx = 100, ny = 100)
)

## Warning in interp(x = latitude, y = depth_m, z = flour_mg_per_m3, duplicate =
## "mean", : collinear points, trying to add some jitter to avoid colinearities!

## Warning in interp(x = latitude, y = depth_m, z = flour_mg_per_m3, duplicate =
## "mean", : success: collinearities reduced through jitter

interp_df <- expand_grid(latitude = interp_grid$x, depth_m = interp_grid$y) %>%
  mutate(flour_mg_per_m3 = as.vector(interp_grid$z))

# interpolate max depth (as a function of latitude) onto the same latitude grid
bathy_interp <- approx(x = max_depth_by_station$latitude,
  y = max_depth_by_station$max_depth,
  xout = interp_grid$x, rule = 2)

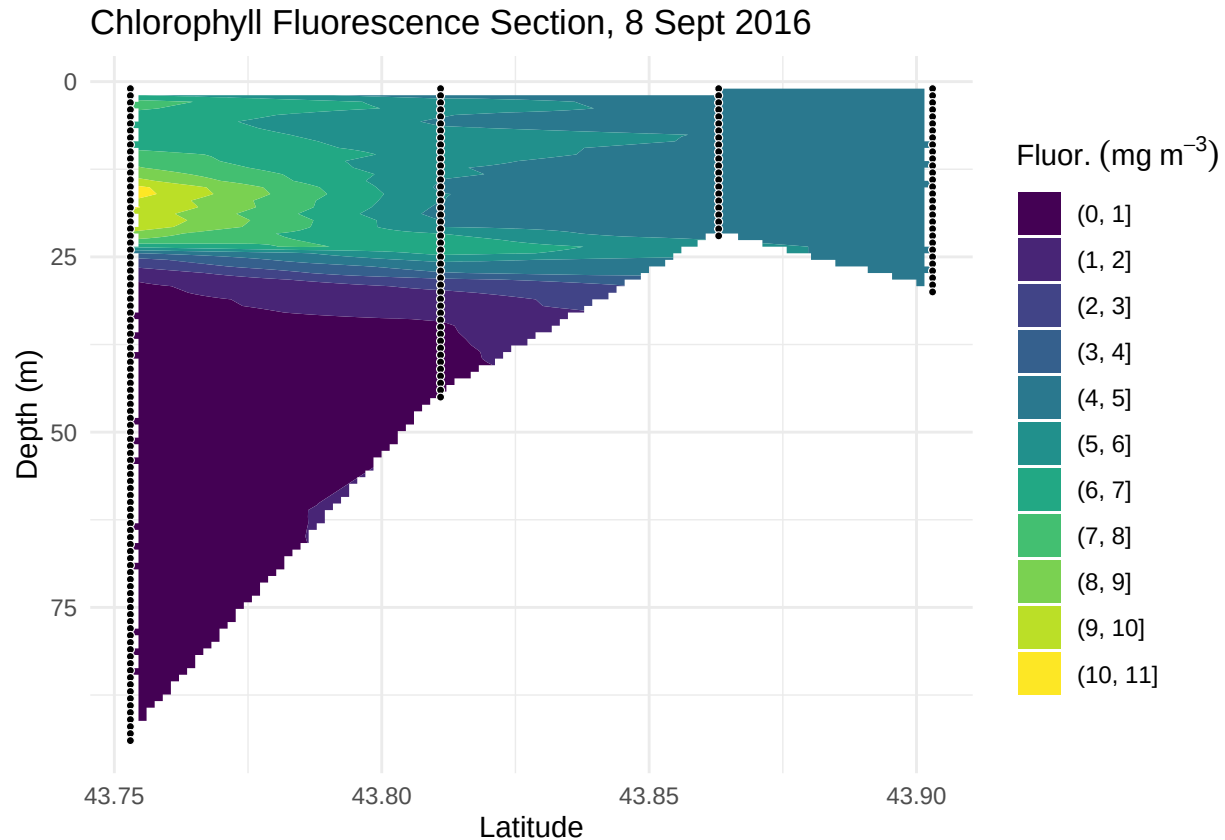
bathy_df <- tibble(latitude = bathy_interp$x, bathy_depth = bathy_interp$y)

# mask: set fluorescence to NA wherever grid depth exceeds the interpolated bathymetry
interp_df_masked <- interp_df %>%
  left_join(bathy_df, by = "latitude") %>%
  mutate(flour_mg_per_m3 = ifelse(depth_m > bathy_depth, NA, flour_mg_per_m3))

ggplot() +
  geom_contour_filled(data = interp_df_masked,
    aes(x = latitude, y = depth_m, z = flour_mg_per_m3),
    na.rm = TRUE) +
  geom_point(data = darts_section,
    aes(x = latitude, y = depth_m,
      color = "white", shape = 21, fill = "black", size = 1.2, stroke = 0.3) +
  scale_y_reverse() +

```

```
scale_fill_viridis_d(name = expression(Fluor.~(mg~m^{-3}))) +
labs(x = "Latitude", y = "Depth (m)",
     title = "Chlorophyll Fluorescence Section, 8 Sept 2016") +
theme_minimal()
```



This looks good for this one cruise - let's scale it up:

Try this prompt: "Can we make a facet plot that does the above, but for all the cruises in 2016"

```
library(purrr)
```

```
station_lat <- tibble(
  station = factor(c(1, 2, 3, 4)),
  latitude = c(43.903, 43.863, 43.811, 43.753)
)

# all 2016 depth-resolved data, with latitude attached
darts_2016_full <- darts %>%
  filter(year == 2016) %>%
  mutate(date = as.Date(date), station = factor(station)) %>%
  left_join(station_lat, by = "station") %>%
  filter(!is.na(fluor_mg_per_m3), !is.na(latitude), !is.na(depth_m))

# function: interpolate + mask a single cruise date's section
```

```

interp_one_cruise <- function(df) {

  # need at least 2 distinct latitudes for akima::interp to work
  if (n_distinct(df$latitude) < 2) return(NULL)

  max_depth_by_station <- df %>%
    group_by(station, latitude) %>%
    summarize(max_depth = max(depth_m, na.rm = TRUE), .groups = "drop") %>%
    arrange(latitude)

  interp_grid <- with(df,
    interp(x = latitude, y = depth_m, z = fluor_mg_per_m3,
      duplicate = "mean", nx = 100, ny = 100)
  )

  interp_df <- expand_grid(latitude = interp_grid$x, depth_m = interp_grid$y) %>%
    mutate(fluor_mg_per_m3 = as.vector(interp_grid$z))

  bathy_interp <- approx(x = max_depth_by_station$latitude,
    y = max_depth_by_station$max_depth,
    xout = interp_grid$x, rule = 2)
  bathy_df <- tibble(latitude = bathy_interp$x, bathy_depth = bathy_interp$y)

  interp_df %>%
    left_join(bathy_df, by = "latitude") %>%
    mutate(fluor_mg_per_m3 = ifelse(depth_m > bathy_depth, NA, fluor_mg_per_m3)) %>%
    select(-bathy_depth)
}

# run per date, tag each result with its date, combine
interp_all <- darts_2016_full %>%
  group_by(date) %>%
  group_split() %>%
  set_names(map_chr(., ~ as.character(.$date[1]))) %>%
  map(interp_one_cruise) %>%
  bind_rows(.id = "date") %>%
  mutate(date = as.Date(date))

```

```

## Warning in interp(x = latitude, y = depth_m, z = fluor_mg_per_m3, duplicate =
## "mean", : collinear points, trying to add some jitter to avoid colinearities!

```

```

## Warning in interp(x = latitude, y = depth_m, z = fluor_mg_per_m3, duplicate =
## "mean", : success: collinearities reduced through jitter

```

```

## Warning in interp(x = latitude, y = depth_m, z = fluor_mg_per_m3, duplicate =
## "mean", : collinear points, trying to add some jitter to avoid colinearities!

```

```

## Warning in interp(x = latitude, y = depth_m, z = fluor_mg_per_m3, duplicate =
## "mean", : success: collinearities reduced through jitter

```

```

## Warning in interp(x = latitude, y = depth_m, z = fluor_mg_per_m3, duplicate =
## "mean", : collinear points, trying to add some jitter to avoid colinearities!

```

```
## Warning in interp(x = latitude, y = depth_m, z = fluor_mg_per_m3, duplicate =
## "mean", : success: collinearities reduced through jitter

## Warning in interp(x = latitude, y = depth_m, z = fluor_mg_per_m3, duplicate =
## "mean", : collinear points, trying to add some jitter to avoid colinearities!

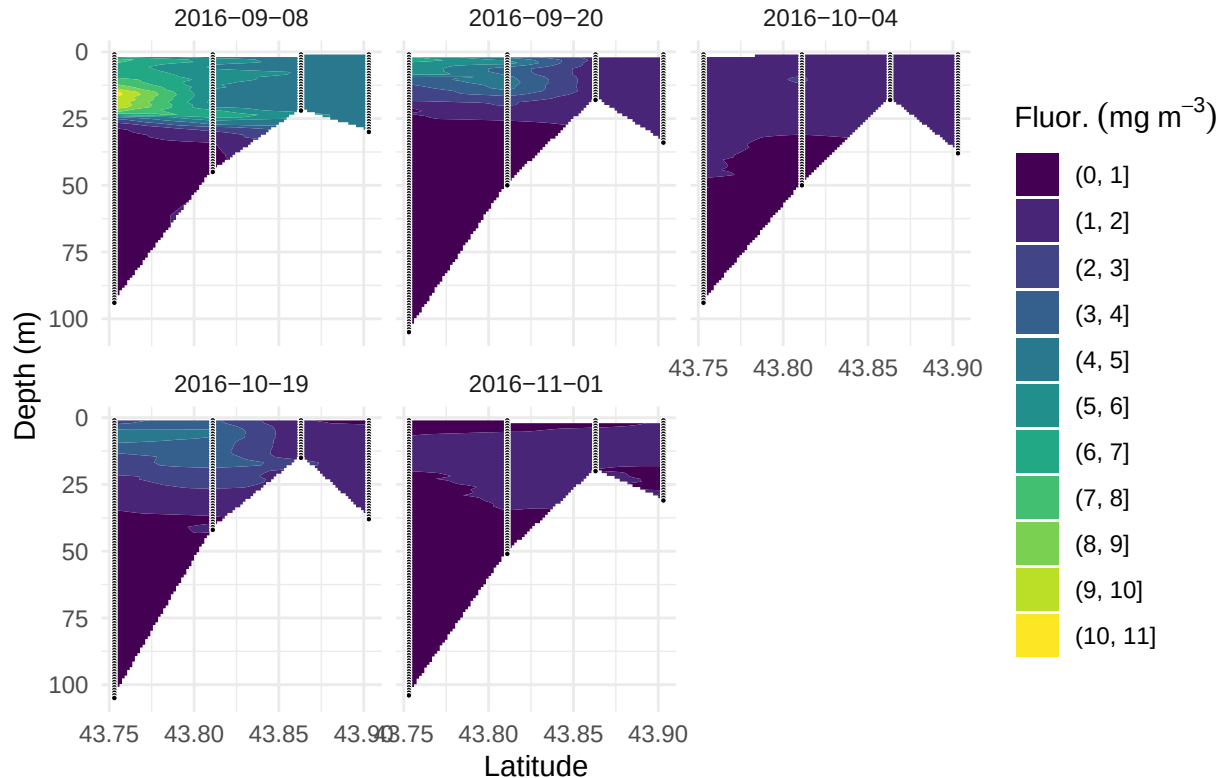
## Warning in interp(x = latitude, y = depth_m, z = fluor_mg_per_m3, duplicate =
## "mean", : success: collinearities reduced through jitter

## Warning in interp(x = latitude, y = depth_m, z = fluor_mg_per_m3, duplicate =
## "mean", : collinear points, trying to add some jitter to avoid colinearities!

## Warning in interp(x = latitude, y = depth_m, z = fluor_mg_per_m3, duplicate =
## "mean", : success: collinearities reduced through jitter
```

```
ggplot() +
  geom_contour_filled(data = interp_all,
                     aes(x = latitude, y = depth_m, z = fluor_mg_per_m3),
                     na.rm = TRUE) +
  geom_point(data = darts_2016_full,
            aes(x = latitude, y = depth_m,
               color = "white", shape = 21, fill = "black", size = 0.8, stroke = 0.3) +
  scale_y_reverse() +
  scale_fill_viridis_d(name = expression(Fluor.~(mg~m^{-3}))) +
  facet_wrap(~ date) +
  labs(x = "Latitude", y = "Depth (m)",
       title = "Chlorophyll Fluorescence Sections, 2016") +
  theme_minimal()
```

Chlorophyll Fluorescence Sections, 2016



Here we've built on the previous exercise and scaled up figure generation quickly: an AI assistant is very helpful for that. But we leaned on the slower, back and forth iterations with the AI assistant to get to one figure style that we liked and was appropriate for our analysis. We've ended up with a large section of code - but because we built it up slowly, it's much easier for us to track what each part is doing, and be happy that it's doing what we want. We could have generated the above figure from the outset with a very detailed prompt - but it would be much harder for us to follow along, and be confident in our result.

e.g.: *"I have an oceanographic dataset in R called darts. Here is the `str(darts)` output. We're going to make a faceted plot in `ggplot2`, where each facet is a different date from 2016. The plots will be `geom_contour_filled`, with `geom_point` for each measurement, with depth on the y, station on the x, and colored by chlorophyll fluorescence. Mask any data interpolated below the depth of a given profile. Use the latitude in place of station number on the x-axis, where stations [1, 2, 3, 4], their latitudes are [43.903, 43.863, 43.811, 43.753]. `str(darts)` output: ..."*

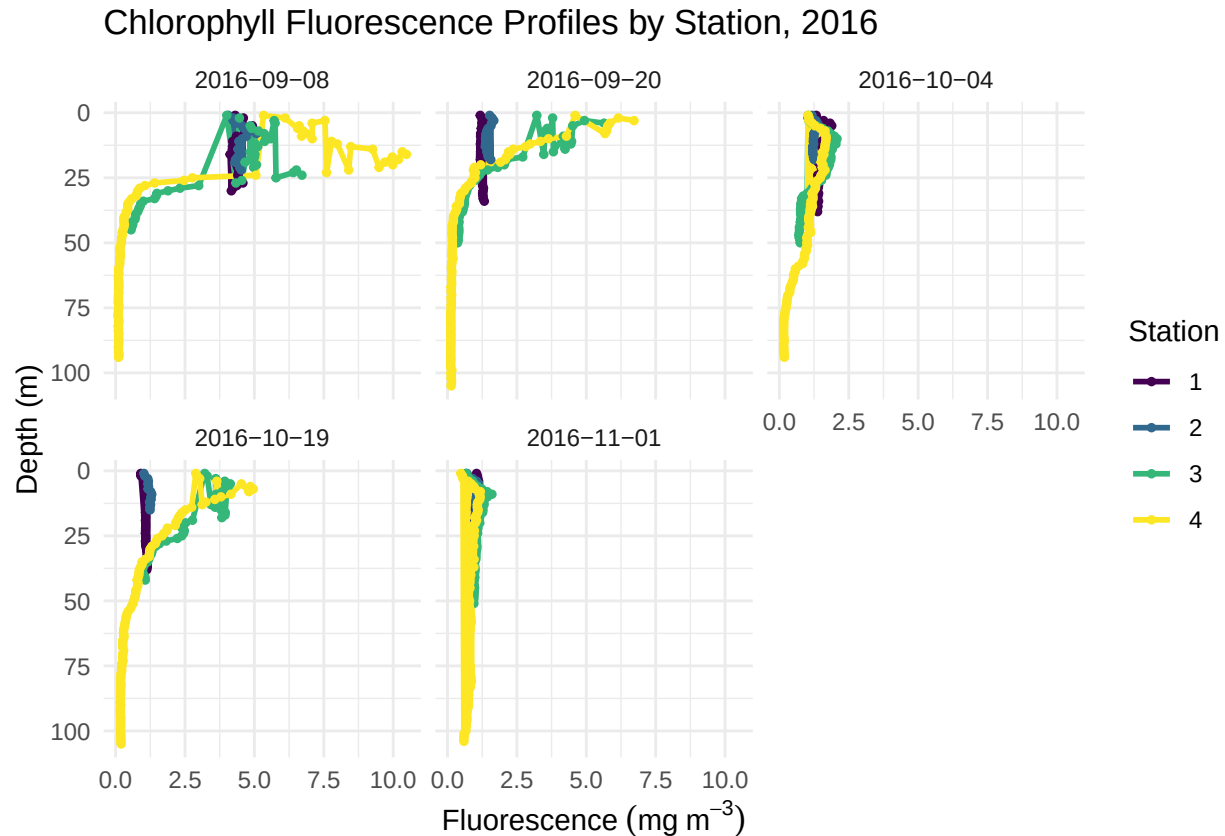
We could also visualize the above data but using line graphs:

Try this prompt: *"Can we do similar to the above (facet by cruise in 2016) but each plot is a line plot with depth on the y, fluorescence on the x and station is a different color"*

```
darts_2016_profiles <- darts %>%
  filter(year == 2016) %>%
  mutate(date = as.Date(date), station = factor(station)) %>%
  filter(!is.na(flueur_mg_per_m3), !is.na(depth_m))

ggplot(darts_2016_profiles, aes(x = flueur_mg_per_m3, y = depth_m, color = station)) +
  geom_line(linewidth = 1) +
```

```
geom_point(size = 1) +
scale_y_reverse() +
scale_color_viridis_d(name = "Station") +
facet_wrap(~ date) +
labs(x = expression(Fluorescence~(mg~m^{-3})), y = "Depth (m)",
     title = "Chlorophyll Fluorescence Profiles by Station, 2016") +
theme_minimal()
```



Which version do you prefer - the line or contour plot? Does it depend on the question you are trying to answer?

Final thoughts and comments on generative AI assisted coding

We've worked through a few different examples, different styles of plots, how to prompt the AI assistant. You can give quite general prompts, sometimes the AI assistant might ask for more details, but sometimes it won't and will do something itself, so it's always best to be as detailed and specific as possible. You can always ask for suggestions on types of plots, but the AI assistant might not find the best type of visualization for your data - having a goal in mind before starting the conversation with the AI assistant is extremely useful and gives you more confidence in the output.

Checklist for evaluating AI-generated plotting code

- ☐ Does it use the correct column names from `ocean_data` (not invented ones)?
- ☐ Are axes labeled with units, not just raw column names?

- ☐ Is the geometry appropriate for the data (e.g., `geom_line` for a true time series, not scattered categorical dates)?
- ☐ If depth is an axis, does it increase downward (`scale_y_reverse()`)?
- ☐ Is the color palette colorblind-safe and appropriate for the data type (`scale_*_viridis_*` for continuous, a qualitative palette for categories)?
- ☐ Does the plot title/caption say what the data actually represents (including that it's simulated, if you're presenting it)?
- ☐ Did you run the code and look at the plot — not just assume it worked?